IEEE INFOCOM 2024 - IEEE Conference on Computer Communications | 979-8-3503-8350-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/INFOCOM52122.2024.10621254

Agglomerative Federated Learning: Empowering Larger Model Training via End-Edge-Cloud Collaboration

Zhiyuan Wu[1,2], Sheng Sun[1], Yuwei Wang[1], Min Liu[1,3], Bo Gao[4], Quyang Pan[1,2], Tianliu He[1,2], and Xuefeng Jiang[1,2]
[1]Institute of Computing Technology, Chinese Academy of Sciences
[2]University of Chinese Academy of Sciences [3]Zhongguancun Laboratory
[4]Beijing Jiaotong University
{wuzhiyuan22s,sunsheng,ywwang,liumin}@ict.ac.cn bogao@bjtu.edu.cn lightinshadow1110@gmail.com
tianliu.he@foxmail.com jiangxuefeng21b@ict.ac.cn

***Abstract*—Federated Learning (FL) enables training Artificial Intelligence (AI) models over end devices without compromising their privacy. As computing tasks are increasingly performed by a combination of cloud, edge, and end devices, FL can benefit from this End-Edge-Cloud Collaboration (EECC) paradigm to achieve collaborative device-scale expansion with real-time access. Although Hierarchical Federated Learning (HFL) supports multi-tier model aggregation suitable for EECC, prior works assume the same model structure on all computing nodes, constraining the model scale by the weakest end devices. To address this issue, we propose Agglomerative Federated Learning (FedAgg), which is a novel EECC-empowered FL framework that allows the trained models from end, edge, to cloud to grow larger in size and stronger in generalization ability. FedAgg recursively organizes computing nodes among all tiers based on Bridge Sample Based Online Distillation Protocol (BSBODP), which enables every pair of parent-child computing nodes to mutually transfer and distill knowledge extracted from generated bridge samples. This design enhances the performance by exploiting the potential of larger models, with privacy constraints of FL and flexibility requirements of EECC both satisfied. Experiments under various settings demonstrate that FedAgg outperforms state-of-the-art methods by an average of 4.53% accuracy gains and remarkable improvements in convergence rate. Our code is available at https://github.com/wuzhiyuan2000/FedAgg.**

***Index Terms*—Federated Learning, End-Edge-Cloud Collaboration, Model Heterogeneity, Interaction Protocol**

I. Introduction

Federated Learning (FL) [1], [2] has emerged as a promising technique for various Artificial Intelligence (AI) applications [3], [4] since it enables collaborative training of AI models over end devices without exchanging their raw data, ensuring both high model performance and data privacy. As the mainstream computing paradigm shifts from centralized cloud computing to decentralized End-Edge-Cloud Collaboration (EECC) [5], large-scale computing tasks are increasingly performed by a combination of centralized cloud servers, bridge edge servers, and a large number of end devices instead of relying solely on the cloud. FL can benefit from this more distributed computing paradigm to fully exploit pervasive devices and rapidly access large amounts of data for delivering real-time services [6]. However, prevailing FL methods [7]–[9] adopt a simple client-server architecture with two tiers (also called levels), where the server is either on the cloud or the edge. The server collaborates with end devices (also called clients) by iteratively aggregating model parameters uploaded from devices and dispatching the updated global model to all devices. These FL methods are not suitable for EECC paradigm, which requires multi-level collaboration among computing nodes.

Fortunately, Hierarchical Federated Learning (HFL) [10] is proposed to support multi-tier model aggregation in FL, from underlying devices to the bridged edge servers and the cloud servers. This manner brings the advantages of device-scale expansion with multi-level collaboration. When implementing HFL with EECC, computing power heterogeneity across end, edge, and cloud nodes should be taken into consideration to fully unleash the potential of EECC. Specifically, cloud servers are equipped with powerful computing capabilities and storage resources, which outperform edge servers, and end devices possess relatively constrained resources [11]. However, existing HFL methods [10], [12]–[14] require imposing the same model structure on all computing nodes for model aggregation, inevitably limiting the scale of trained models by the weakest end devices due to the bottleneck effect and encountering resource under-utilization over edge and cloud computing nodes. To take full advantage of the computation superiority of edge and cloud nodes over end devices, it is critical to deploy appropriate models on end, edge, and cloud nodes that are compatible with their computation capability and also achieve model-agnostic [15] collaborative training in EECC-empowered FL methods. In this way, larger models can be trained on the edge and cloud nodes with stronger characterization and generalization ability to achieve performance improvement.

To this end, we propose a novel FL framework suitable for EECC paradigm, named Agglomerative Federated Learning (FedAgg). To be specific, FedAgg recursively organizes computing nodes to collaboratively train models that grow in size from end devices, edge servers to cloud servers via our customized Bridge Sample Based Online Distillation Protocol (BSBODP), which defines the interaction rules among every

This paper has been accepted by IEEE International Conference on Computer Communications. *(Corresponding author: Yuwei Wang)*

pair of parent-child computing nodes in an end-edge-cloud network with a tree-topology. During training, the knowledge acquired by the upper-level nodes (closer to the cloud) from the lower-level nodes (closer to devices) is recursively transferred to the cloud, tier by tier via BSBODP. In this way, upper-level nodes can deploy larger models with higher capability and integrate the knowledge of lower-level nodes in an agglomerative manner. Regarding BSBODP, model-agnostic collaborative training is achieved in any pair of parent-child computing nodes, where a pre-trained lightweight decoder is used to generate fake samples (called bridge samples) whose extracted logits are used for knowledge distillation among nodes. To our best knowledge, **agglomerative federated learning is the first framework empowered by end-edge-cloud collaboration paradigm that enables training larger models with ever-increasing capability tier by tier up to the cloud.** The proposed framework is able to train models that are adapted to the power of computing nodes and satisfy the privacy constraints of FL as well as the flexibility requirements of EECC.

The main contributions of this paper are summarized as follows:

- We propose a novel FL framework suitable for EECC paradigm, named Agglomerative Federated Learning (FedAgg), which recursively organizes computing nodes via a customized interaction protocol and enables models trained on the end, edge, and cloud nodes to grow larger in size and stronger in generalization ability.
- We design the Bridge Sample Based Online Distillation Protocol (BSBODP) to achieve model-agnostic collaborative training among interacted nodes via online distillation over generated bridge samples.
- We validate the effectiveness of FedAgg in an end-edge-cloud architecture over CIFAR-10 and CIFAR-100 datasets. Empirical results demonstrate that FedAgg achieves superior accuracy and significantly faster convergence compared with related state-of-the-art methods.

II. Related Work

A. Hierarchical Federated Learning

Hierarchical Federated Learning (HFL) is first proposed by Liu [10], where the established end-edge-cloud FL architecture combines the advantages of both massive data coverage of cloud computing and low communication latency of edge computing. Following Liu [10], a series of works [12]–[14] are proposed to accelerate the training process or improve the system performance of HFL by building clients' cluster topology and performing hierarchical aggregation on top of local training, taking into account the computing and communication capability [12], [13] as well as data distribution and mobility [14] of end devices. In particular, Nguyen [16] proposes a novel hierarchical distributed learning framework inspired by democratized learning [17], which extends conventional HFL through a self-organizing hierarchical structure based on agglomerative clustering, and enhances both local personalization and global generalization in distributed training. However, all the above methods require participating computing nodes to adopt the same model structure, which limits the model scale to the capability of end devices and wastes the resources of edge and cloud nodes with substantial computing power.

B. Training Larger Model in Federated Learning

With sufficient training data, increasing model size is a common way of improving model accuracy and generalization ability in deep learning [18]. However, prevailing FL approaches [7]–[9] require homogeneous models to be adopted in both the powerful central server and the resource-limited end devices, which prevents the training of larger models beyond the maximum capability of end devices. To address this challenge, prior works [19]–[24] propose two types of solutions to train larger models that surpass the resource limits of devices, one based on partial training and the other based on knowledge distillation. In particular, partial training-based approaches [19], [20] divide the complete global model into multiple sub-models and then train these sub-models on multiple clients in parallel, thereby allowing the model on server obtained by collaborative training to exceed the size of the largest model on clients. Knowledge distillation-based approaches [21]–[24] leverage the model output of clients as a regularization term for the server-side model training, thereby achieving the transfer of integrated representation learned by clients on decentralized private data. However, the above methods are designed for two-tier architectures and cannot support collaborative model training empowered by end-edge-cloud networks. Therefore, the applicability of the aforementioned works is highly restricted.

III. Preliminary

A. End-Edge-Cloud Collaboration

End-Edge-Cloud Collaboration (EECC) is an emerging computing paradigm that coordinates multi-level heterogeneous computing nodes including central cloud servers, numerous edge servers, and massively distributed end devices, aiming to collaboratively work on large-scale computing tasks. In general, end devices are at the periphery of the network, generating data to perform data-relevant AI applications. Edge servers bridge to connect end devices and cloud servers, distributed at intermediate locations along the network. Cloud servers are equipped with sufficient computation and storage resources, enabling the coordination of underlying end devices and edge servers.

We consider a typical End-Edge-Cloud Network (EEC-NET) with a tree topology (shown in Fig. 1) denoted as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of computing nodes and $\mathcal{E}$ is the set of communication links. The entire EEC-NET G has only one root node $r \in \mathcal{V}$ and contains one or more leaf nodes that form a set $\mathcal{L} \subset \mathcal{V}$. Each computing node $v \in \mathcal{V}$ except leaf nodes has one or more children, and all of which form a set $Child(v)$. Any computing node $v \in \mathcal{V}$ except the root node has one parent $Parent(v)$. Besides, we define $Leaf(v)$ to be all the leaf nodes of the sub-tree that seeks the computing node v as its root, and hence we have $Leaf(r) = \mathcal{L}$. Without loss of generality, the computing nodes in EEC-NET are arranged by levels. Assuming a EEC-NET G have a total of T tiers, and the computing nodes in tier $t \in \{1, 2, ..., T\}$ form a set $\mathcal{V}_t$, so that we have $\mathcal{V}_T = \mathcal{L}$ and $\mathcal{V}_1 = \{r\}$. For flexibility reasons, EECC should take into

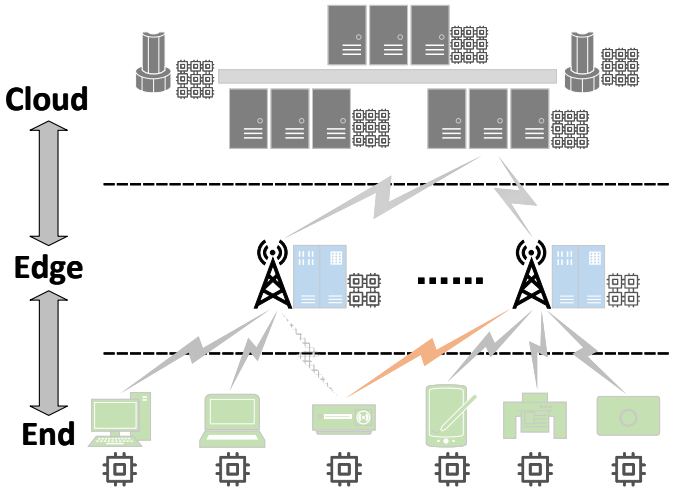


Fig. 1. Framework of end-edge-cloud collaboration.

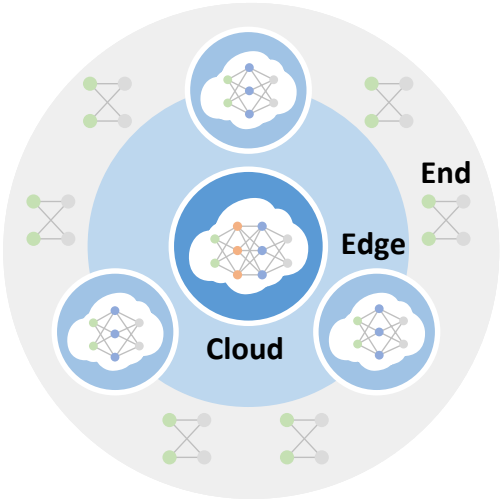


Fig. 2. Expected model scales on different tiers in an end-edge-cloud network.

account the dynamic migration of computing nodes caused by manifold factors such as uneven load distribution, unreliable network connections, node failures, etc. Specifically, each node should be able to flexibly switch to different parent nodes at the same level, as shown in Fig. 1.

B. Hierarchical Federated Learning Empowered by End-Edge-Cloud Collaboration

Assume $K = \mid \mathcal{L} \mid$ end devices (called clients) participate in Hierarchical Federated Learning (HFL) over an EEC-NET, and each client $k \in \{1, 2, ..., K\}$ owns a local dataset $\mathcal{D}^k = \bigcup_{i=1}^{N^k} \{(X_i^k, y_i^k)\}$, in which N^k is the number of samples in $\mathcal{D}^k$, and X_i^k, y_i^k are the data and label of the i-th sample in $\mathcal{D}^k$, respectively. Besides, each node $v \in \mathcal{V}$ holds a model $Model(v)$ with parameters W^v, and enables to exchange information with its parent and child nodes, i.e., $\{Parent(v)\} \cup Child(v)$. Our goal is to obtain the model in the root node $Model(r)$ that minimizes its training loss $L_{train}(\cdot)$ over all private data on devices, that is:

$$\underset{W^r}{\operatorname{argmin}} L_{train}(\bigcup_{k=1}^{K} \mathcal{D}^k; W^r). \tag{1}$$

To fully exploit heterogeneous resources and accommodate the differentiated capabilities of end, edge, and cloud computing nodes, the edge and cloud servers with powerful computation resources should deploy larger models than end devices to enhance performance by capturing more complex and generalized patterns from private data, while the model size on end nodes with limited capability should be reduced accordingly, as shown in Fig. 2.

C. Interaction Protocols in Federated Learning

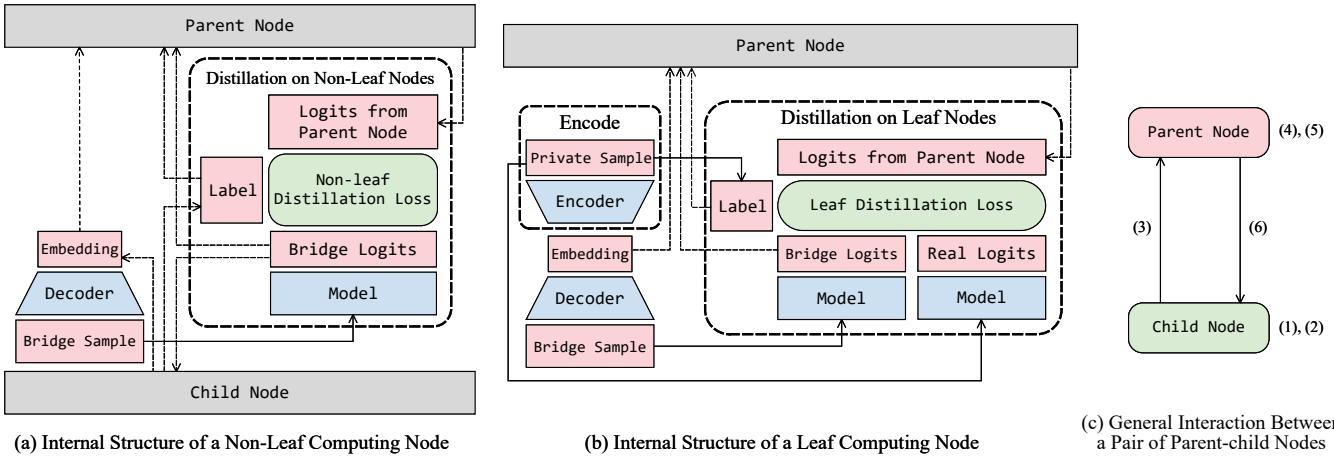
Interaction protocols in FL are rules of information exchange between computing nodes in an FL system, including rules for both content transfer and parameters update. Prevailing FL interaction protocols are based on parameters interaction like FedAvg [7], where upper-level nodes aggregate and distribute model parameters from and to lower-level nodes. However, this protocol requires all computing nodes to deploy a model with the same structure, which limits the model size on the cloud or the edge by the capacity of end devices.

IV. Agglomerative Federated Learning

A. Motivation and Overview

Considering the intrinsic property of EECC, lower-level end devices possess abundant data but limited computing resources, while the upper-level nodes have powerful computation ability but no data. In order to access local data directly and meet the privacy requirements of FL, existing EECC-empowered FL methods [10], [12]–[14] conduct model training on end devices and only perform model aggregation on the edge and cloud. However, this manner requires the same model structure to be adopted among all computing nodes, thereby limiting the scale of trained models by the weakest end devices and encountering computation capability waste of edge and cloud nodes.

To exploit the powerful computation of edge and cloud nodes, we prompt the intuitive motivation of training larger models on edge and cloud nodes than end devices for achieving higher accuracy with stronger generalization abilities. A feasible way to realize this idea is to conduct online distillation [25], [26] as an interaction protocol among computing nodes at different tiers, which enables different nodes to train models with heterogeneous structures via iteratively exchanging the logits (also called knowledge) between two nodes to guide reciprocal model training. However, directly integrating online distillation into EECC-powered FL may raise privacy concerns about sharing data on devices, as the same sample is required to extract logits across different computing nodes. To overcome this, we design a new interaction protocol named Bridge Sample Based Online Distillation Protocol (BSBODP), which employs fake data as a bridge to transfer knowledge across multi-level nodes with the assistance of a lightweight pre-trained autoencoder. This fake data is generated by the



(a) Internal Structure of a Non-Leaf Computing Node

(b) Internal Structure of a Leaf Computing Node

(c) General Interaction Between a Pair of Parent-child Nodes

Fig. 3. Overview of BSBODP. (1) Child Node Distillation on Bridge Samples. (2) Logits Extraction on Child Node. (3) Upload Logits to the Parent Node. (4) Parent Node Distillation on Bridge Samples. (5) Logits Extraction on Parent Node. (6) Distribute Logits to the Child Node.

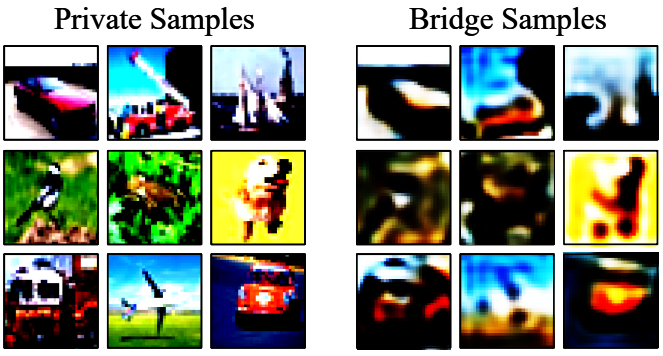


Fig. 4. Comparison of private samples and bridge samples.

decoder from either encoded or received embeddings, allowing for computing logits without revealing raw data on devices.

Given a multi-tier architecture in EECC, each node can adopt a model structure that matches its own computing power, and the lower-level nodes can iteratively transfer their learned representations to the upper-level nodes via BSBODP. Therefore, the model on the cloud can eventually integrate the knowledge of all nodes and achieve satisfactory performance with sufficient capacity. Based on this concept, we propose Agglomerative Federated Learning (FedAgg), which enables the training of larger models with better performance on powerful cloud and edge nodes than on resource-limited end devices via agglomerating knowledge generated by ever-expanding models from bottom to top in the multi-tier architecture in EECC. By leveraging the advantages of knowledge agglomeration, FedAgg enables the trained models from the bottom to the top to grow in size and generalization ability without violating the privacy principle in FL. Moreover, FedAgg can also handle scenarios where any computing nodes dynamically change their parents within the same tier, which ensures deployment flexibility in a realistic EEC-NET.

B. Bridge Sample Based Online Distillation Protocol

Fig. 3 illustrates the overview of our designed BSBODP, where each computing node preserves a pre-trained decoder and a conventional complete model (the model to be trained on the computing node). In particular, each leaf computing node additionally preserves a pre-trained encoder (forms a pre-trained autoencoder with the aforementioned decoder) for encoding local data into embeddings, which will be transmitted to its parent node until reaching the root node. During the execution phase, each computing node utilizes the decoder to generate bridge samples that match the data distribution of its corresponding leaf nodes based on the embeddings encoded from private data or uploaded by sub-nodes. These bridge samples are used as intermediaries to conduct online distillation between every pair of parent-child computing nodes aligned by the same embedding. As illustrated in Fig. 4, it is difficult to recover raw information from the embeddings of private samples since the embeddings are generated based on an extremely lightweight autoencoder (<50K model parameters) that are pre-trained on a super large open dataset (such as ImageNet [27]). As the autoencoder is not able to capture the fine-grained features and reconstruct training samples from them, bridge sample generation and distillation will not reveal data privacy.

We formulate the process of BSBODP as follows: the decoder on each computing node is defined as $dec(\cdot)$, and the encoder on each leaf computing node is defined as $enc(\cdot)$. In addition, the model to be trained on any computing node $Model(v^*), v^* \in \mathcal{V}$ can give an inference on the input data via $f(\cdot; W^{v^*})$. When performing upward knowledge distillation, the child node in any pair of parent-child computing nodes acts as the teacher v^T, and the parent node acts as the student v^S. When performing downward knowledge distillation, the roles are reversed. Specifically, each model on non-leaf student computing node $Model(v^S), \forall v^S \notin \mathcal{L}$ distills the knowledge from the model on its teacher $Model(v^T)$ over the bridge samples that are generated from the embeddings ε corresponding to $\bigcup_{u \in Leaf(v^S)} \mathcal{D}^u \cap \bigcup_{u \in Leaf(v^T)} \mathcal{D}^u$. With generated bridge

Algorithm 1: Bridge Sample Based Online Distillation Protocol (BSBODP)

Input: v^1,v^2
Output: Trained $Model(v^1)$, $Model(v^2)$

procedure BSBODP(v^1, v^2)
  BSBODP-DIRECTIONAL(v^1, v^2)
  BSBODP-DIRECTIONAL(v^2, v^1)
  return $Model(v^1)$, $Model(v^2)$
end procedure
procedure BSBODP-DIRECTIONAL(v^S, v^T)
  v^T generates bridge samples $dec(\varepsilon)$ from all stored embeddings ε
  v^T extracts logits $z^{\varepsilon} = f(dec(\varepsilon); W^{v^T})$ on bridge samples, and transmits the results to v^S
  if $v^S \notin \mathcal{L}$ **then**
    Optimize W^{v^S} according to Eq. (2)
  end
  else
    Optimize W^{v^S} according to Eq. (4)
  end
end procedure

samples as intermediates, $Model(v^S)$ is optimized according to the non-leaf distillation loss as follows:

$$\begin{aligned}
&\min_{W^{v^S}} L_{non-leaf} \\
&= \min_{W^{v^S}} [L_{CE}(\tau(f(dec(\varepsilon); W^{v^S})), y^{\varepsilon}) + \\
&\beta \cdot KL(\tau(f(dec(\varepsilon); W^{v^S}))||\tau(\tfrac{z^{\varepsilon}}{T}))] \\
&= \min_{W^{v^S}} [L_{CE}(\tau(f(dec(\varepsilon); W^{v^S})), y^{\varepsilon}) + \\
&\beta \cdot KL(\tau(f(dec(\varepsilon); W^{v^S}))||\tau(\underbrace{\frac{f(dec(\varepsilon); W^{v^T})}{T}}_{\frac{z^{\varepsilon}}{T}}))],
\end{aligned} \tag{2}$$

where $L_{CE}(\cdot)$ is the cross-entropy loss function, $KL(\cdot)$ is the Kullback-Leibler divergence loss function, and β is the distillation weight. Moreover, y^{ε} and z^{ε} are the labels and extracted logits of bridge samples corresponding to embeddings ε uploaded from child nodes, and ε is ultimately generated at the leaf nodes according to the following equation:

$$\varepsilon = enc(X^*), \forall (X^*, y^*) \in \bigcup_{u \in Leaf(v^S)} \mathcal{D}^u \cap \bigcup_{u \in Leaf(v^T)} \mathcal{D}^u. \tag{3}$$

Moreover, each model on leaf student computing node $Model(v^S), \forall v^S \in \mathcal{L}$ is optimized subject to a linear combination of the non-leaf distillation loss over generated bridge samples and the local training losses (controlled by a weight γ) over private samples $(X^*, y^*) \in \mathcal{D}^{v^S}$, that is:

$$\begin{aligned}
&\min_{W^{v^S}} L_{leaf} \\
&= \min_{W^{v^S}} L_{CE}(f(X^*; W^{v^S}), y^*) + \gamma \cdot L_{non-leaf},
\end{aligned} \tag{4}$$

where

$$\varepsilon = enc(X^*), \forall (X^*, y^*) \in \mathcal{D}^{v^S}. \tag{5}$$

Once the above steps are completed, v^S and v^T need to swap their roles and optimize in the opposite direction following the above constraints, as shown in Algorithm 1. Thereout, the models to be trained on every pair of parent-child computing nodes can learn from each other, and the representation learned through knowledge distillation can be propagated to the model on the cloud tier by tier, starting from the leaf nodes.

Algorithm 2: Agglomerative Federated Learning (FedAgg)

Input: $\mathcal{V}, \mathcal{E}$
Output: Trained $Model(v^*), \forall v^* \in \mathcal{V}$

procedure FEDAGG($\mathcal{V}, \mathcal{E}$)
  INIT($r, \mathcal{E}$)
  while *do not reach maximum epoches* **do**
  FEDAGGTRAIN($r, \mathcal{E}$)
  end
end procedure
procedure INIT($v^*, \mathcal{E}$)
  if $v^* = r$ **then**
  for $u \in Child(v^*)$ *in parallel* **do**
    INIT($u, \mathcal{E}$)
    Receive and store embeddings ε with corresponding labels from u
  end
  end
  else if $v^* \in \mathcal{L}$ **then**
  Extract and store embeddings ε via $enc(X^*), \forall X^* \in \mathcal{D}^{v^*}$
  Transmit embeddings ε with corresponding labels to $Parent(v^*)$
  end
  else
  for $u \in Child(v^*)$ *in parallel* **do**
    INIT($u, \mathcal{E}$)
    Receive and store embeddings ε from u
    Send embeddings ε with corresponding labels to $Parent(v^*)$
  end
  end
end procedure
procedure FEDAGGTRAIN($v^*, \mathcal{E}$)
  if $v^* = r$ **then**
  for $u \in Child(v^*)$ *in parallel* **do**
    FEDAGGTRAIN($u, \mathcal{E}$)
  end
  end
  else if $v^* \in \mathcal{L}$ **then**
  BSBODP(v^*,$Parent(v^*)$)
  end
  else
  for $u \in Child(v^*)$ *in parallel* **do**
    FEDAGGTRAIN($u, \mathcal{E}$)
    BSBODP(v^*,$Parent(v^*)$)
  end
  end
end procedure

C. Recursive Agglomeration in End-Edge-Cloud Networks

Considering FL in an EEC-NET where computing nodes are organized in a tree topology, we propose the FedAgg framework, which applies BSBODP to achieve collaborative training over every pair of parent-child computing nodes, recursively distilling knowledge from bottom to top in an agglomerative manner, as formulated in Algorithm 2. Specifically, FedAgg includes two main phases: initialization and recursive training with BSBODP. In the initialization phase, leaf nodes generate embeddings using their pre-trained encoders and send them up the tree hierarchy to the root node. In the training phase, each

node recursively distills the knowledge from the sub-tree that seeks it as the root, and passes the newly extracted knowledge to its parent, applying BSBODP to enable interaction between parent-child pairs. This process starts from the leaf nodes and ends with the cloud server, where the model on the cloud is updated with the learned knowledge from all tiers. Due to the nature of EEC-NET's computing resources and the model-agnostic property of BSBODP, the upper-level nodes can deploy larger models and integrate more knowledge from lower-level nodes to achieve superior performance, and the largest model (i.e. the model on the cloud) is exactly the model that we want to gain through collaborative training.

D. Deployment Flexibility Guarantees of Agglomerative Federated Learning

FedAgg supports the dynamic migration of computing nodes to guarantee deployment flexibility in an EEC-NET, where various factors such as load balancing, connection unreliability, and node failures, may require computing nodes to switch to a different parent at the same level. To demonstrate the advantage of FedAgg in handling dynamic migration of computing nodes, we classify existing FL interaction protocols that potentially support hierarchical collaborative model training into two types based on their constraints on model structures of the interacting computing nodes: equivalence interaction protocols and partial order interaction protocols. In the following part, we will give formal definitions of these two types of interaction protocols, and prove that equivalence interaction protocols, including our proposed BSBODP adopted by FedAgg, provide better support for dynamic migration of computing nodes.

Definition 1. *(Equivalence Interaction Protocol). Consider an EEC-NET with a tree topology formulated in section III-A, we define a binary relation R over any pair of parent-child computing nodes. The sufficient necessary condition for an equivalence interaction protocol is defined as follows:*

$$< Model(v^i), Model(v^i) >\in R, \forall v^i \in \mathcal{V}, \tag{6}$$

$$\begin{aligned} &< Model(v^i), Model(v^j) >\in R \wedge v^i \neq v^j \\ &\rightarrow < Model(v^j), Model(v^i) >\in R, \forall v^i, v^j \in \mathcal{V}, \end{aligned} \tag{7}$$

$$\begin{aligned} &< Model(v^i), Model(v^j) >\in R \\ &\wedge < Model(v^j), Model(v^k) >\in R \\ &\rightarrow < Model(v^i), Model(v^k) >\in R, \forall v^i, v^j, v^k \in \mathcal{V}, \end{aligned} \tag{8}$$

that is:

$$Model(v^i) \sim Model(v^j), \forall < v^i, v^j >\in \mathcal{E}. \tag{9}$$

According to our definition, we can easily prove that the followings are equivalent interaction protocols: 1) the same model structure is adopted on the parent and child computing nodes, i.e. $Model(v^i) = Model(v^j), \forall < v^i, v^j >\in \mathcal{E}$, which is represented by FedAvg [7]; 2) model-agnostic interaction protocols that do not impose restrictions on the model structures of parent and child computing nodes, i.e. $Model(v^i) \perp Model(v^j), \forall < v^i, v^j >\in \mathcal{E}$, which is represented by BSBODP.

Definition 2. *(Partial Order Interaction Protocol). Consider an EEC-NET with a tree topology formulated in section III-A, we define a binary relation R over any pair of parent-child computing nodes. The sufficient necessary condition of a partial order interaction protocol is defined as follows:*

$$< Model(v^i), Model(v^i) >\in R, \forall v^i \in \mathcal{V}, \tag{10}$$

$$\begin{aligned} &< Model(v^i), Model(v^j) >\in R \\ &\wedge < Model(v^j), Model(v^i) >\in R \\ &\rightarrow v^i = v^j, \forall v^i, v^j \in \mathcal{V}, \end{aligned} \tag{11}$$

$$\begin{aligned} &< Model(v^i), Model(v^j) >\in R \\ &\wedge < Model(v^j), Model(v^k) >\in R \\ &\rightarrow < Model(v^i), Model(v^k) >\in R, \forall v^i, v^j, v^k \in \mathcal{V}, \end{aligned} \tag{12}$$

that is:

$$Model(v^i) \preceq Model(v^j), \forall < v^i, v^j >\in \mathcal{E}. \tag{13}$$

We can also prove that partial training-based interaction protocols [19], [20], which require the model on the child node to be a sub-model of that on the parent node, i.e. $Model(v^i) \subseteq Model(Parent(v^i)), \forall < v^i, Parent(v^i) >\in \mathcal{E}$ are partial order interaction protocols.

Theorem 1. *HFL methods based on equivalence interaction protocols allow the parent of any v^1 switch to $Parent(v^2)$, where $Parent(v^1)$ and $Parent(v^2)$ are at the same level, i.e. $Model(v^1) \sim Model(Parent(v^2)), \forall v^1, v^2 \in \mathcal{V}_t, t \in \{2, 3, ..., T\}$.*

Proof.

Case 1.1. When $v^1, v^2 \in \mathcal{V}_2$, we have:

$$Parent(v^1) = Parent(v^2) = r. \tag{14}$$

Hence, v^1 and v^2 have the same parent node, and there is no dynamic migration of computing nodes in this case.

Case 1.2. When $v^1, v^2 \in \mathcal{V}_t, t \geq 3$, we first define

$$Parent^n(\cdot) = \underbrace{Parent(Parent(\dots Parent(\cdot)\dots))}_{\text{call } Parent(\cdot) \text{ for } n \text{ times}}, \tag{15}$$

then, we have:

$$\begin{aligned} &Model(Parent(v^1)) \sim Model(Parent^2(v^1)) \sim ... \\ &\sim Model(Parent^{t-1}(v^1)) = r, \end{aligned} \tag{16}$$

and

$$\begin{aligned} &Model(Parent(v^2)) \sim Model(Parent^2(v^2)) \sim ... \\ &\sim Model(Parent^{t-1}(v^2)) = r. \end{aligned} \tag{17}$$

And then,

$$\begin{aligned} &Model(v^1) \sim Model(Parent(v^1)) \\ &\sim r \sim Model(Parent(v^2)), \end{aligned} \tag{18}$$

that is:

$$Model(v^1) \sim Model(Parent(v^2)). \tag{19}$$

Therefore, the computing node v^1 is allowed to become a child of $Parent(v^2)$, i.e. dynamic migration of computing nodes at the same level is allowed. □

Theorem 2. *HFL methods based on partial order interaction protocols do not necessarily allow the parent of any v^1 switch to $Parent(v^2)$, where $Parent(v^1)$ and $Parent(v^2)$ are at the same level, i.e. $\neg Model(v^1) \preceq Model(Parent(v^2)), \exists v^1, v^2 \in \mathcal{V}_t, t \in \{2, 3, ..., T\}$.*

Proof.

Case 2.1. When $v^1, v^2 \in \mathcal{V}_2$, we have:

$$Parent(v^1) = Parent(v^2) = r. \tag{20}$$

Hence, v^1 and v^2 have the same parent node r, and there is no dynamic migration of computing nodes in this case.

Case 2.2. When $v^1, v^2 \in \mathcal{V}_t, t \geq 3$, there are two sub-cases:

$$Model(Parent(v^1)) \preceq Model(Parent(v^2)), \tag{21}$$

and

$$Model(Parent(v^2)) \preceq Model(Parent(v^1)). \tag{22}$$

When Eq. (22) is satisfied, there exists a situation where computing node v^1 is not allowed to switch its parent to $Parent(v^2)$. Instantiating $\preceq$ to the partial order relation over integers $\leq$, we construct a tree topology $10(9(8,7),5(4,3))$ and set function $Model(\cdot)$ to be a constant function, i.e. $Model(x) = x, \forall x$. In our setting, the parent of 7 (v^1) and 3 (v^2) are at the same level and $Model(Parent(3)) = 5 \leq 9 = Model(Parent(7))$ satisfies Eq. (22). At this point, $\neg Model(7) \leq Model(Parent(3))$ is equivalent to $\neg 7 \leq 5$, which is apparently true. Hence, dynamic migration of computing nodes at the same level is not necessarily allowed. □

V. Experiments

A. Experiment Setup

1) Experimental Setting: Our experiments are conducted based on FedML [28], which is a research library for FL. We use the common CIFAR-10 and CIFAR-100 datasets [29], which are partitioned into K non-independent and identically distributed parts as private datasets of K different devices. The number of clients is set as $K \in \{225, 400\}$ and $K \in \{144, 196, 225\}$ on CIFAR-10 and CIFAR-100 datasets, respectively. Following [24], we adopt a hyper-parameter α pre-set up in FedML to control the degree of data heterogeneity among devices. In our experiments, we set the values of $\alpha \in \{1.0, 3.0\}$ corresponding to the high and low degrees of data heterogeneity, respectively. In terms of network topology, we consider a three-tier tree-structured EEC-NET in our experiments, from bottom to top are end devices, edge nodes, and cloud nodes, where the number of edge servers is set to $\sqrt{K}$ and only one cloud server is considered. Each end device is either coordinated by an edge server or connected directly to the cloud server. Regarding model structures, we deploy the same lightweight 6-layer autoencoder pre-trained on ImageNet [27] on all the end devices and the decoder part of the autoencoder on other computing nodes. In addition, 3-layer-Convolutional Neural Networks (3-layer-CNN), ResNet-10 and ResNet-18 [30] with small to large model sizes are deployed on the end, edge, and cloud nodes respectively, fitting the capability of computing nodes across different tiers. The configurations of model structures can be found in TABLE I.

2) Baselines and Criteria: We compare our proposed FedAgg with four state-of-the-art algorithms, HierFAVG [10] and DemLearn [16], which support collaborative model training among multi-tier architectures; FedGKT [22] and FedDKC [24], which enables training larger models on the server side than that on the client side in a two-tier architecture. In our experiments, we assume that the servers in FedGKT and FedDKC are deployed on the cloud and directly communicate with end devices. In addition, system performance is evaluated by the test accuracy of the cloud-side model after convergence or within 500 communication rounds.

TABLE I
Model configurations on CIFAR10 dataset. The output dimension of the last fully connected layer needs to be adjusted to 20 on CIFAR100.

Model Structure	**Notation**	**Feature Extractor**	**Classifier**
3-Layer -CNN	M^1	9.31K	10.88K
ResNet-10	M^2		4.67M
ResNet-18	M^3		10.65M
Model Structure	**Notation**	**Encoder**	**Decoder**
6-Layer -Autoencoder	-	1.90K	2.47K

3) Hyperparameters: We conduct the learning rate of 0.001 with batch size 8 to all methods. Besides, personalized hyper-parameters are set as follows:

- HierFAVG adopts $\kappa_1 = 1$ and $\kappa_2 = 1$.
- DemLearn adopts the default hyper-parameters setting in [31].
- FedGKT adopts $\beta = 1.5$.
- FedDKC adopts KKR as the knowledge refinement strategy with $\beta = 1.5$ and $T = 0.12$.
- FedAgg adopt $\gamma = 1$, $T = 3$ and $\beta = 10$.

B. Performance Evaluation

1) Comparison of Test Accuracy: We compare FedAgg with HierFAVG, DemLearn, FedGKT, and FedDKC under different numbers of clients and data heterogeneity settings on CIFAR-10, and the results are shown in TABLE II. We can see that FedAgg achieves the highest accuracy in all settings, which outperforms the highest accuracy among HierFAVG, DemLearn, FedGKT, and FedDKC by 4.33%, 1.74%, 5.04% and 4.52% in experimental settings from left to right. These results not only show the better performance of FedAgg, but also demonstrate its ability to handle different data distributions and the varying number of clients. To further confirm the superiority of FedAgg under different organizing structures of clients, we compare the performance of FedAgg with HierFAVG and DemLearn on CIFAR-100 under varying numbers of hierarchical structure levels ($lv \in \{3, 5, 7, 10\}$) adopted in DemLearn. As shown in TABLE III, FedAgg outperforms the baseline algorithms in all experimental settings over CIFAR-100, which outperforms the highest accuracy among HierFAVG and DemLearn with varying hierarchical structures by 1.32%, 2.39%, 1.69% respectively in terms of 144, 196 and 225 clients. This demonstrates that FedAgg can benefit from exploiting the potential of deploying larger models on edge and cloud nodes in an EEC-NET, and is suitable for scenarios with different numbers of devices.

2) Convergence Rate: We evaluate the convergence rate of FedAgg against baseline algorithms on both CIFAR-10 and CIFAR-100 datasets from two perspectives: the required

TABLE II
TEST ACCURACY (%) ON CIFAR-10 DATASET.

Method	Model			225 Clients		400 Clients		Average
	End	Edge	Cloud	$\alpha = 3.0$	$\alpha = 1.0$	$\alpha = 3.0$	$\alpha = 1.0$	
HierFAVG	M^1	M^1		10.15	18.35	22.25	18.73	17.37
DemLearn				30.55	32.13	29.25	28.06	30.00
FedGKT		-	M^3	27.39	26.59	30.54	29.22	28.44
FedDKC				32.43	30.09	30.79	28.78	30.52
FedAgg		M^2		**36.76**	**33.87**	**35.83**	**33.74**	**35.05**

TABLE III
TEST ACCURACY (%) ON CIFAR-100 DATASET, TAKING $\alpha = 3.0$.

Method	Model			144 Clients	196 Clients	225 Clients	Average
	End	Edge	Cloud				
HierFAVG	M^1	M^1		9.11	9.42	5.03	7.85
DemLearn (lv=3)				15.75	14.94	14.81	15.17
DemLearn (lv=5)				14.99	14.59	14.55	14.71
DemLearn (lv=7)				14.52	13.97	13.89	14.13
DemLearn (lv=10)				12.56	13.25	12.52	12.78
FedAgg		M^2	M^3	**17.07**	**17.33**	**16.50**	**16.97**

TABLE IV
COMMUNICATION ROUNDS WHEN REACHING A GIVEN TEST ACCURACY ON CIFAR-10 DATASET. COMMUNICATION ROUNDS ARE COUNTED FROM 0, THE SAME AS BELOW.

Method	225 Clients				400 Clients			
	$\alpha = 3.0$		$\alpha = 1.0$		$\alpha = 3.0$		$\alpha = 1.0$	
	25%	30%	25%	30%	25%	30%	25%	30%
HierFAVG	-	-	-	-	-	-	-	-
DemLearn	138	458	113	333	216	-	243	-
FedGKT	12	-	23	-	6	18	11	-
FedDKC	8	14	12	23	7	24	12	-
FedAgg	**0**	**2**	**1**	**7**	**1**	**4**	**5**	**10**

TABLE V
COMMUNICATION ROUNDS WHEN REACHING A GIVEN TEST ACCURACY ON CIFAR-100 DATASET, TAKING $\alpha = 3.0$.

Method	144 Clients		196 Clients		225 Clients	
	9%	14%	9%	14%	9%	14%
HierFAVG	17	-	13	-	-	-
DemLearn (lv=3)	33	266	64	365	28	350
DemLearn (lv=5)	36	347	69	386	34	436
DemLearn (lv=7)	46	425	64	-	30	-
DemLearn (lv=10)	33	-	50	-	31	-
FedAgg	**0**	**5**	**0**	**12**	**0**	**7**

communication rounds when reaching a given test accuracy, and the learning curves. TABLE IV and V show the communication rounds needed to reach a given test accuracy on CIFAR-10 and CIFAR-100 datasets, respectively. As shown in the two tables, FedAgg requires much fewer communication rounds to converge to a given test accuracy than all compared algorithms, thus demonstrating remarkable improvements in convergence rate. Besides, Fig. 5 shows the learning curves under different degrees of data heterogeneity and varying numbers of clients on CIFAR-10, while Fig. 6 shows the learning curves under different organizing structures on CIFAR-100. From Fig. 5 and Fig. 6, we can conclude that FedAgg consistently outperforms the baseline algorithms in terms of test accuracy with the same number of communication rounds, and also guarantees faster convergence under various experimental settings.

VI. ABLATION STUDY

A. Effectiveness of Online Distillation

TABLE VI shows the impact of the distillation weight β on the performance of the cloud-side model. We can observe that when β is set to a relatively small value ($\beta = 0.1$), the performance of the cloud-side model is poor, which indicates the effectiveness of transferring knowledge from lower-level computing nodes through online distillation. However, when β is set to a very large value ($\beta = 50$), the performance of the cloud-side model drops significantly due to the excessive weakening of cross-entropy-based optimization.

B. Tolerance to Device Heterogeneity

To examine the impact of heterogeneous on-device models on the system performance, we modify the model structure of 20% clients from M^1 to M^2, and compare their experimental results with those of all clients using the same model structure M^1, as shown in TABLE VII. The results demonstrate that the variation of the model structures on clients has a negligible effect on the performance of the cloud-side model. This implies that FedAgg can tolerate the heterogeneity of device-side models and is suitable for the EECC scenario where devices have different computing capabilities.

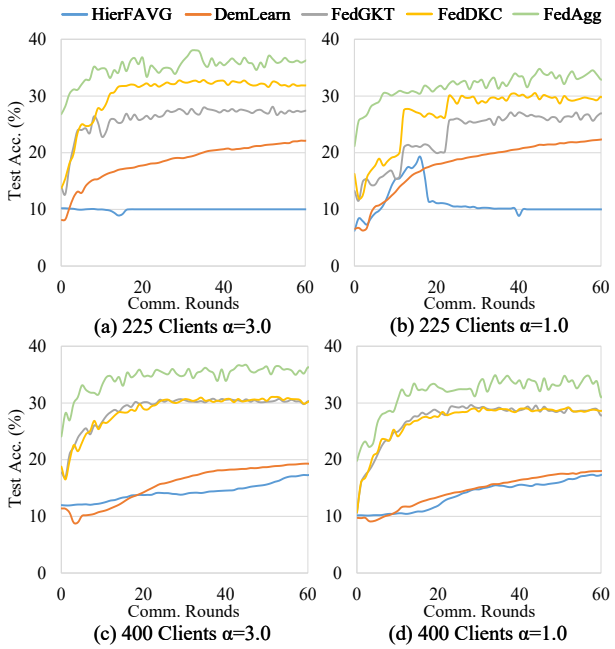


Fig. 5. Learning curves of the cloud-side model with varying numbers of devices and degrees of data heterogeneity. Results are obtained on CIFAR-10 dataset.

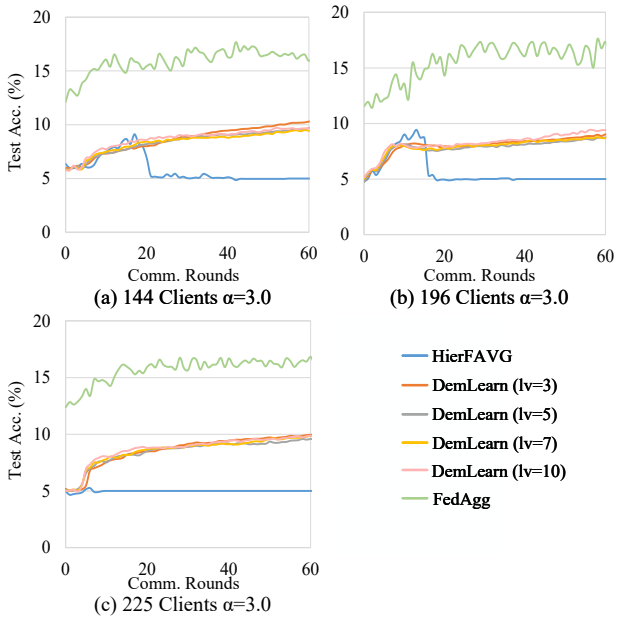


Fig. 6. Learning curves of the cloud-side model with varying numbers of devices. Results are obtained on CIFAR-100 dataset.

C. Smaller Cloud-side Models

We compare the performance of FedAgg using smaller cloud-side models M^1 and M^2 with that obtained by using a larger cloud model M^3, as shown in TABLE VIII. We observe that the performance of FedAgg drops significantly as the model size on the cloud decreases. This further confirms that our proposed FedAgg can achieve remarkable performance gains by enabling the deployment of larger models on the cloud than on end devices.

TABLE VI
FEDAGG WITH DIFFERENT β. EXPERIMENTS ARE CONDUCTED ON CIFAR-10 DATASET WITH 225 CLIENTS, TAKING $\alpha = 1.0$.

Method	$\beta = 0.1$	$\beta = 1$	$\beta = 3$	$\beta = 10$	$\beta = 50$
FedAgg	33.14	**35.33**	34.66	33.87	27.21

TABLE VII
FEDAGG WITH DIFFERENT MODEL HETEROGENEITY SETUP ON DEVICES. EXPERIMENTS ARE CONDUCTED ON CIFAR-10 DATASET WITH 225 CLIENTS, TAKING $\alpha = 1.0$.

Method	Model Homo.	Model Hetero.
FedAgg	**33.87**	32.83

TABLE VIII
FEDAGG WITH DIFFERENT MODEL STRUCTURES ON THE CLOUD. EXPERIMENTS ARE CONDUCTED ON CIFAR-10 DATASET WITH 225 CLIENTS, TAKING $\alpha = 1.0$.

Method	3-Layer-CNN	ResNet-10	ResNet-18
FedAgg	23.84	31.11	**33.87**

VII. CONCLUSION

To overcome the limitation of model scale on powerful edge and cloud computing nodes constrained by the weakest end devices in EECC-empowered FL, we propose a novel federated learning framework called Agglomerative Federated Learning (FedAgg), which allows the trained models from end devices, bridge edge servers, to cloud servers to grow larger in size and stronger in generalization ability. To be specific, FedAgg recursively distills knowledge from bottom to top in an agglomeration manner via our customized Bridge Sample Based Online Distillation Protocol (BSBODP), which can achieve model-agnostic interaction across every pair of parent-child computing nodes through online distillation over generated bridge samples. To our best knowledge, FedAgg is the first framework empowered by EECC that enables training larger models with ever-increasing capability tier by tier up to the cloud, and can achieve superior performance than related state-of-the-art methods in terms of test accuracy and convergence rate.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2021YFB2900102), the National Natural Science Foundation of China (No. 62072436), the Innovation Capability Support Program of Shaanxi (No. 2023-CX-TD-08), Shaanxi Qinchuangyuan "scientists+engineers" team (No. 2023KXJ-040), the Innovation Funding of ICT, CAS (No. E261080). We thank Zeju Li from Beijing University of Posts and Telecommunications, Sijie Cheng from Tsinghua University, Tian Wen, Wen Wang, and Yufeng Chen from Institute of Computing Technology, Chinese Academy of Sciences, and Jinda Lu from the University of Science and Technology of China for inspiring suggestions.

REFERENCES

[1] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[3] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for internet of things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1622–1658, 2021.

[4] T. Zhang, L. Gao, C. He, M. Zhang, B. Krishnamachari, and A. S. Avestimehr, "Federated learning for the internet of things: Applications, challenges, and opportunities," *IEEE Internet of Things Magazine*, vol. 5, no. 1, pp. 24–29, 2022.

[5] S. Duan, D. Wang, J. Ren, F. Lyu, Y. Zhang, H. Wu, and X. Shen, "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Communications Surveys & Tutorials*, 2022.

[6] G. Bao and P. Guo, "Federated learning in cloud-edge collaborative architecture: key technologies, applications and challenges," *Journal of Cloud Computing*, vol. 11, no. 1, p. 94, 2022.

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[8] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.

[9] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.

[10] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020, pp. 1–6.

[11] A. Tak and S. Cherkaoui, "Federated edge learning: Design issues and challenges," *IEEE Network*, vol. 35, no. 2, pp. 252–258, 2020.

[12] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Transactions on Mobile Computing*, 2022.

[13] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.

[14] C. Feng, H. H. Yang, D. Hu, Z. Zhao, T. Q. Quek, and G. Min, "Mobility-aware cluster federated learning in hierarchical wireless networks," *IEEE Transactions on Wireless Communications*, vol. 21, no. 10, pp. 8441–8458, 2022.

[15] A. Afonin and S. P. Karimireddy, "Towards model agnostic federated learning using knowledge distillation," *CoRR*, vol. abs/2110.15210, 2021. [Online]. Available: https://arxiv.org/abs/2110.15210

[16] M. N. Nguyen, S. R. Pandey, T. N. Dang, E.-N. Huh, N. H. Tran, W. Saad, and C. S. Hong, "Self-organizing democratized learning: Toward large-scale distributed learning systems," *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[17] M. N. Nguyen, S. R. Pandey, K. Thar, N. H. Tran, M. Chen, W. S. Bradley, and C. S. Hong, "Distributed and democratized learning: Philosophy and research challenges," *IEEE Computational Intelligence Magazine*, vol. 16, no. 1, pp. 49–62, 2021.

[18] X. Hu, L. Chu, J. Pei, W. Liu, and J. Bian, "Model complexity of deep learning: A survey," *Knowledge and Information Systems*, vol. 63, pp. 2585–2619, 2021.

[19] S. Alam, L. Liu, M. Yan, and M. Zhang, "Fedrolex: Model-heterogeneous federated learning with rolling sub-model extraction," in *Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[20] Q. Nguyen, H. H. Pham, K.-S. Wong, P. Le Nguyen, T. T. Nguyen, and M. N. Do, "Feddct: Federated learning of large convolutional neural networks on resource constrained devices using divide and collaborative training," *IEEE Transactions on Network and Service Management*, 2023.

[21] Y. J. Cho, A. Manoel, G. Joshi, R. Sim, and D. Dimitriadis, "Heterogeneous ensemble knowledge transfer for training large models in federated learning," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, L. D. Raedt, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2022, pp. 2881–2887, main Track. [Online]. Available: https://doi.org/10.24963/ijcai.2022/399

[22] C. He, M. Annavaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large cnns at the edge," *Advances in Neural Information Processing Systems*, vol. 33, pp. 14 068–14 080, 2020.

[23] S. Cheng, J. Wu, Y. Xiao, and Y. Liu, "Fedgems: Federated learning of larger server models via selective knowledge fusion," *arXiv preprint arXiv:2110.11027*, 2021.

[24] Z. Wu, S. Sun, Y. Wang, M. Liu, Q. Pan, J. Zhang, Z. Li, and Q. Liu, "Exploring the distributed knowledge congruence in proxy-data-free federated distillation," *ACM Trans. Intell. Syst. Technol.*, dec 2023, just Accepted. [Online]. Available: https://doi.org/10.1145/3639369

[25] R. Anil, G. Pereyra, A. Passos, R. Ormandi, G. E. Dahl, and G. E. Hinton, "Large scale distributed neural network training through online distillation," *arXiv preprint arXiv:1804.03235*, 2018.

[26] L. Wang and K.-J. Yoon, "Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.

[27] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.

[28] C. He, S. Li, J. So, X. Zeng, M. Zhang, H. Wang, X. Wang, P. Vepakomma, A. Singh, H. Qiu *et al.*, "Fedml: A research library and benchmark for federated machine learning," *arXiv preprint arXiv:2007.13518*, 2020.

[29] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[31] M. N. Nguyen, S. R. Pandey, T. N. Dang, E.-N. Huh, N. H. Tran, W. Saad, and C. S. Hong. [Online]. Available: https://github.com/nhatminh/Dem-AI